Glowy

Architecture et code expliqués

Projet d'équipe · octobre 2026

Ce document explique **comment le code est organisé et pourquoi**, puis suit une vraie requête (une réservation) à travers chaque couche, avec le code réel du projet. À lire avant de modifier le code.

Voir aussi : `GESTIONS.md` (ce que fait chaque partie), `API.md` (les routes), `GUIDE-EQUIPE.md` (lancement, Next.js, GSAP).

Vue d'ensemble

Glowy est **un seul projet Next.js** qui contient à la fois le site (les pages) et le serveur (l'API). Il n'y a pas de backend séparé.

```
Navigateur / téléphone (PWA)
├─ pages rendues par le serveur (HTML)          ← src/app/**/page.tsx
├─ composants interactifs (React côté client)   ← "use client"
├─ service worker : cache hors ligne, push      ← public/sw.js
│
│  fetch("/api/...")  (JSON)
▼
Serveur Next.js
├─ routes API        src/app/api/**/route.ts    (contrôle d'accès, validation)
├─ modules métier    src/modules/<module>/      (règles, cas d'usage)
│     domain  →  application  →  infrastructure
├─ bus d'événements  src/infrastructure/event-bus.ts
▼
MongoDB  (Mongoose, src/infrastructure/db/models.ts)
```

Couche technique	Technologie
Langage	TypeScript (strict) partout
Site et API	Next.js 16 (App Router), React 19
Style	Tailwind CSS 4, composants shadcn / Base UI
Base de données	MongoDB + Mongoose 9
Authentification	Auth.js v5 (session JWT)
Validation	Zod 4
Tests	`node:test` + `tsx`, base MongoDB dédiée

Les dossiers

```
src/
├─ app/                     Next.js : une URL = un dossier
│  ├─ (site)/               site public + espace client /compte
│  ├─ (auth)/               connexion, inscription, mot de passe oublié
│  ├─ dashboard/            espace du salon
│  ├─ admin/                espace Super Admin
│  ├─ api/                  routes REST (route.ts)
│  ├─ layout.tsx            mise en page racine (polices, langue, PWA)
│  └─ manifest.ts           manifest de la PWA
├─ modules/                 le métier, un dossier par domaine
│  ├─ identity/  booking/  catalog/  reviews/  loyalty/  …
│  ├─ subscribers.ts        branchement des événements
│  └─ scheduler.ts          tâches planifiées
├─ core/domain/             règles communes : erreurs, rôles, heure de Tunis, événements
├─ infrastructure/          technique partagée : db/, event-bus, http, session
├─ components/              composants d'interface (ui/, salon/, booking/, manage/, pwa/…)
├─ i18n/                    traductions fr / ar / en et formatage
├─ lib/                     utilitaires (appels API côté client, pagination, GSAP…)
├─ auth.ts, auth.config.ts  configuration Auth.js
└─ proxy.ts                 redirections rapides selon le rôle
public/sw.js                service worker
tests/                      tests automatisés
scripts/seed.ts             données de démonstration
```

Les parenthèses `(site)`, `(auth)` regroupent des pages sans changer l'URL : `(site)/salons/page.tsx` répond à `/salons`.

Un module, trois couches

Chaque domaine métier est un module de `src/modules/`. Exemple avec `booking/` (réservation) :

```
src/modules/booking/
├─ domain/            règles pures : aucune base, aucun Next.js
│  ├─ policy.ts       acompte, délais, transitions de statut, tranches de 5 min
│  └─ availability.ts calcul des créneaux libres
├─ application/       cas d'usage
│  ├─ schemas.ts      validation Zod des données reçues
│  ├─ ports.ts        interfaces dont les cas d'usage ont besoin
│  └─ use-cases.ts    createBooking, cancelBooking, rescheduleBooking…
├─ infrastructure/    accès à MongoDB
│  ├─ mongo-booking-repository.ts   implémente les ports avec Mongoose
│  └─ slot-locks.ts   verrou anti double réservation
└─ index.ts           API publique du module + câblage
```

La règle de dépendance : `domain` ne dépend de rien ; `application` dépend de `domain` et de **ses propres interfaces (ports)** ; `infrastructure` implémente ces interfaces avec MongoDB ; `index.ts` branche le tout. Les pages, les routes et les autres modules n'importent **que** `@/modules/<module>` (le fichier `index.ts`).

Pourquoi ce découpage ?

- **Testable** : les règles de `domain` se testent sans base de données ; les cas d'usage peuvent recevoir une fausse horloge.
- **Remplaçable** : changer de base (on est passé de PostgreSQL à MongoDB) ne touche que `infrastructure/`.
- **Lisible** : chaque personne de l'équipe sait où se trouve chaque chose.

Les ports (interfaces)

Le cas d'usage ne connaît pas MongoDB : il demande un `BookingRepository`.

```
// src/modules/booking/application/ports.ts
export interface BookingRepository {
  salonBySlug(slug: string): Promise<{ id: string; active: boolean; depositPercent: number } | null>;
  activeServices(salonId: string, keys: string[]): Promise<ServiceRef[]>;
  /** Verrouille les tranches de l'employé puis enregistre la réservation ; null si une tranche est déjà prise */
  insertIfFree(data: NewBooking): Promise<BookingRecord | null>;
  cancel(id: string, at: Date, refund: boolean): Promise<BookingRecord | null>;
  // …
}
```

Le câblage (composition)

`index.ts` fabrique le module en lui donnant ses dépendances réelles :

```
// src/modules/booking/index.ts
export const createBookingModule = (overrides: { clock?: Clock } = {}) => {
  const clock = overrides.clock ?? systemClock;
  return makeBooking({
    bookings: mongoBookingRepository,          // MongoDB
    access: { canManageSalon },                // module Identité
    promotions: createPromotionsModule({ clock }),
    offPeak: { hours: offPeakHoursFor },       // IA d'affluence
    events: eventBus,                          // bus d'événements
    clock,                                     // horloge (remplaçable en test)
  });
};

export const { createBooking, cancelBooking, rescheduleBooking /* … */ } = createBookingModule();
```

Suivre une requête : une cliente réserve

1. Le navigateur appelle l'API

Le tunnel de réservation (composant client) envoie la demande avec l'utilitaire `api()` de `src/lib/api.ts`, qui renvoie `{ data }` ou lève une `ApiError` avec un code stable (`SLOT_TAKEN`, `EMAIL_TAKEN`…) traduit à l'écran.

```
await api("/api/bookings", { method: "POST", body: { salonSlug, serviceKeys, employee, date, time, paymentMethod } });
```

2. La route API : accès et validation

```
// src/app/api/bookings/route.ts
export const POST = route(async (request) => {
  const user = await requireUser("CLIENT");                    // connecté ET rôle cliente
  const input = await readJson(request, createBookingInput);   // validation Zod
  const booking = await createBooking(user.id, input);         // cas d'usage
  return ok(booking, { status: 201 });
});
```

- `route()` (`src/infrastructure/http.ts`) refuse une écriture venant d'un autre site (**CSRF**) et transforme toute erreur métier en réponse JSON `{ error: { code, message } }` avec le bon statut HTTP.
- `requireUser()` (`src/infrastructure/session.ts`) relit le compte **en base** : un compte bloqué perd l'accès tout de suite, même avec une session encore valide.

- `createBookingInput` (Zod) rejette toute donnée mal formée avant qu'elle n'atteigne le métier :

```
export const createBookingInput = z.object({
  salonSlug: z.string().min(1),
  serviceKeys: z.array(z.string().min(1)).min(1).max(6),
  employee: z.string().default("any"),
  date: DATE,
  time: TIME,
  paymentMethod: z.enum(["ONLINE", "ONSITE"]).default("ONSITE"),
  promoCode: z.string().trim().max(20).optional(),
});
```

3. Le cas d'usage : la logique métier

```
// src/modules/booking/application/use-cases.ts (extrait)
async function createBooking(clientId: string, input: CreateBookingInput) {
  const salon = await bookings.salonBySlug(input.salonSlug);
  if (!salon?.active) throw notFound("Salon introuvable.");

  const { services, durationMin, total } = await resolveServices(salon.id, input.serviceKeys);
  const startsAt = tunisToUtc(input.date, input.time);                // heure de Tunis → UTC
  if (startsTooSoon(startsAt, now)) throw badRequest("SLOT_IN_PAST", "Ce créneau est déjà passé.");

  for (const employee of freeEmployees(day, minutes, durationMin)) {   // « peu importe » : premier libre
    const booking = await bookings.insertIfFree({
      clientId, salonId: salon.id, employeeId: employee.id, startsAt, endsAt,
      deposit: depositFor(price, input.paymentMethod, salon.depositPercent),
      offPeak: isOffPeak(day.weekday, minutes, forecastHours),         // heures creuses prévues par l'IA
      /* … */
    });
    if (booking) {
      await events.publish("booking.created", { bookingId: booking.id });
      return booking;
    }
  }
  throw conflict("SLOT_TAKEN", "Ce créneau n'est plus disponible. Choisissez-en un autre.");
}
```

Les règles viennent de `domain/` (fonctions pures, testées) :

```
// src/modules/booking/domain/policy.ts
/** Acompte : 20 % par défaut, borné entre 10 et 30 DT, jamais au-delà du total */
export const depositFor = (total: number, method: PaymentMethod, percent = 20) =>
  method === "ONLINE" && total > 0 && percent > 0 ? Math.min(total, 30, Math.max(10, Math.round((total * percent) / 100))) : 0;
```

4. L'infrastructure : le verrou anti double réservation

C'est la base de données qui garantit qu'un créneau n'est jamais réservé deux fois, même si deux clientes cliquent **à la même milliseconde**. Chaque réservation insère une ligne `SlotLock` par tranche de 5 minutes occupée ; un **index unique** `(employé, tranche)` refuse la seconde insertion.

```
// src/modules/booking/infrastructure/slot-locks.ts
export async function acquireSlices(employee, start, end, owner) {
  const docs = lockSlices(start, end).map((at) => ({ employee, at, ...owner }));
  try {
    await SlotLock.insertMany(docs, { ordered: true });
    return true;
  } catch (error) {
    if (!isDuplicateKey(error)) throw error;                 // E11000 : tranche déjà prise
    await SlotLock.deleteMany({ ...owner, at: { $in: docs.map((d) => d.at) } });
    return false;                                            // → le cas d'usage essaie un autre employé, sinon 409
  }
}
```

Un test vérifie que **10 réservations simultanées** du même créneau donnent **une seule** réservation (`tests/booking.test.ts`).

5. Les événements : les autres modules réagissent

Le cas d'usage publie `booking.created`. Il ne sait pas qui écoute. Les abonnés sont branchés dans `src/modules/subscribers.ts` :

```
// src/modules/loyalty/application/subscribers.ts (extrait)
bus.on("booking.completed", async ({ bookingId }) => {
  const booking = await facts.completedBooking(bookingId);
  if (!booking || (await loyalty.alreadyAwarded(booking.clientId, "BOOKING_COMPLETED", { bookingId }))) return;
  for (const grant of rewardsForCompletedBooking(booking)) await loyalty.award(grant.userId, grant.reason, grant);
});
```

Événement	Publié par	Qui réagit
`user.registered`	identity	notifications (bienvenue)
`salon.registered`	identity, dashboard	notifications, subscription
`salon.statusChanged`	admin	notifications (salon validé / suspendu)
`booking.created`	booking	notifications (confirmation cliente et salon)
`booking.cancelled`	booking	notifications, fidélité (annulation tardive), liste d'attente
`booking.completed`	booking	fidélité (points), promotions (points du salon)
`booking.noShow`	booking	fidélité (pénalité)
`review.published`	reviews	fidélité (points d'avis), notifications
`plan.paymentRequested` / `plan.paymentDecided`	subscription	notifications

Le contrat de chaque événement est typé dans `src/core/domain/events.ts`. Une erreur d'un abonné est journalisée **sans annuler** l'action d'origine. Pour ajouter une réaction : ajouter un abonné, **sans modifier** le module qui publie.

Les pages : serveur et client

Next.js distingue deux sortes de composants.

	Composant serveur (par défaut)	Composant client (`"use client"`)
S'exécute	sur le serveur	dans le navigateur
Peut	lire la base via les modules, vérifier la session	gérer l'état, les clics, les formulaires
Ne peut pas	utiliser `useState`, `onClick`	importer du code serveur (modules, base)
Exemples	`page.tsx`, `layout.tsx`	tunnel de réservation, formulaires, carte

Une page serveur charge ses données directement, sans passer par l'API :

```
// src/app/dashboard/services/page.tsx
export default async function ServicesPage() {
  const user = await requirePageUser("MANAGER", "ADMIN");          // redirige si non autorisé
  const salon = await staffSalon(user);
  const [{ t }, services] = await Promise.all([getI18n(), listSalonServices(salon.id)]);
  return <ServicesManager services={services} />;                  // composant client pour l'interaction
}
```

Le composant client modifie ensuite les données via l'API (`api("/api/dashboard/services", …)`), puis `router.refresh()` recharge les données de la page serveur.

Particularités de Next.js 16 utilisées dans le projet :

- `params` et `searchParams` sont des promesses : `const { slug } = await params;`
- type `PageProps<"/salons/[slug]">` et `RouteContext<"/api/…">` générés par Next ;
- le fichier `proxy.ts` remplace l'ancien `middleware.ts`.

La sécurité, couche par couche

Où	Quoi
`src/proxy.ts`	redirection rapide selon le rôle (`/admin` réservé au Super Admin…)
`requireUser()` / `requirePageUser()`	contrôle définitif dans **chaque** route et page, compte relu en base
`route()`	refus des écritures venant d'un autre site (CSRF)
Zod	validation de toutes les entrées
`identity/domain/account.ts`	blocage 15 min après 5 échecs ; mots de passe hachés (bcrypt)
`appOrigin()`	liens envoyés par e-mail construits depuis `APP_URL` (pas depuis l'en-tête `Host`)
`AuditLog`	journal des actions sensibles du Super Admin
`public/sw.js`	aucune donnée privée en cache après la déconnexion

La base de données

Tous les modèles sont dans `src/infrastructure/db/models.ts` (33 collections). Principes :

- **Une collection par agrégat** (`User`, `Salon`, `Booking`, `Review`…).
- **Imbrication** de ce qui vit toujours avec le parent : horaires, photos et prestations dans `Salon` ; lignes et paiement dans `Booking` ; compte fidélité et favoris dans `User`.
- **Index** sur les recherches fréquentes (`{ salon, startsAt }`, `{ client, startsAt }`) et **index uniques** pour les règles qui ne doivent jamais être violées (e-mail, verrou des créneaux).
- **Transactions** (`withTransaction`) pour les opérations en plusieurs écritures (inscription d'un salon) quand MongoDB tourne en replica set ; sinon, les index uniques et les mises à jour atomiques protègent les données.
- Seules les couches `infrastructure/` importent `@/infrastructure/db`.

Les langues (i18n)

- La langue est dans le cookie `NEXT_LOCALE` (`fr`, `ar`, `en`), sans préfixe dans l'URL.
- `src/i18n/dictionaries/fr.ts` est la **source de vérité** : le type `Dictionary = typeof fr` oblige `en.ts` et `ar.ts` à avoir toutes les clés (erreur TypeScript sinon).
- Côté serveur : `const { t, f } = await getI18n()` ; côté client : `const { t, f } = useI18n()`. `f("Page {page} sur {pages}", { page, pages })` remplit les variables.
- En arabe, la page passe en `dir="rtl"` ; les icônes directionnelles ont `rtl:rotate-180`.

Animations, PWA, tâches planifiées

- **GSAP** : un élément avec `data-animate="fade-up"` (ou `split`, `stagger`…) est masqué par le CSS puis révélé par `MotionRoot`. Les règles sont dans `design-system/MASTER.md`.
- **PWA** : `src/app/manifest.ts` rend l'application installable ; `public/sw.js` gère le cache hors ligne (pages consultées et préchargées, scripts, images) et les notifications push. Hors ligne testable seulement en production (`npm run build && npm start`).
- **Tâches planifiées** (`src/modules/scheduler.ts`) : toutes les 10 minutes, rappels de rendez-vous, préavis puis expiration des points, analyse des avis. Chaque tâche est **idempotente** (la relancer ne crée pas de doublon).

Les tests

- `npm test` : 75 tests sur la base `MONGODB_URI_TEST`, vidée avant chaque test.
- Règles pures testées sans base (`tests/pagination.test.ts`, `recommend`, `affluence`…), parcours complets testés avec la base (`booking`, `loyalty-reviews`, `password-reset`…).
- Exemple de test d'une règle critique :

```
// tests/booking.test.ts
test("10 réservations simultanées du même créneau : une seule réussit", async () => {
  await makeSalon();
  const clients = await Promise.all(Array.from({ length: 10 }, () => makeUser()));
  const results = await Promise.allSettled(clients.map((c) => createBooking(c.id, input())));
  assert.equal(results.filter((r) => r.status === "fulfilled").length, 1);
  assert.equal(results.filter((r) => r.status === "rejected").length, 9);
});
```

Ajouter une fonctionnalité : la recette

1. **Modèle** : ajouter ou compléter la collection dans `src/infrastructure/db/models.ts`.
2. **Règle** : fonction pure dans `src/modules/<module>/domain/` + son test.
3. **Cas d'usage** : schéma Zod dans `application/schemas.ts`, logique dans `application/use-cases.ts` (ou une fonction dans `infrastructure/` pour une simple requête).
4. **Accès base** : dans `infrastructure/`.
5. **Exporter** depuis `src/modules/<module>/index.ts`.
6. **Route API** : `src/app/api/.../route.ts` avec `route()`, `requireUser()`, `readJson(schema)`.
7. **Interface** : page serveur qui lit les données + composant client qui appelle l'API.
8. **Textes** : clés dans `fr.ts`, `en.ts` et `ar.ts`.

9. **Vérifier** : `npx tsc --noEmit`, `npm run lint`, `npm test`, puis l'écran à 375 px et en arabe.